StockSense Pro

Production-Ready · Multi-Tenant · Cloud-Native · Event-Driven

Java + Spring Boot | MySQL + Redis | Apache Kafka | Kubernetes | Multi-Tenant SaaS | Event-Driven

Table of Contents

1. Executive Summary & Vision

StockSense Pro is a cloud-native, multi-tenant, event-driven inventory and point-of-sale management SaaS platform engineered to serve retail businesses of all types — from school uniform garment shops to grocery stores, medical stores, and large format retail. The platform leverages barcode scanning, real-time Kafka event streams, Redis caching, and a Kubernetes-based microservices backend to deliver sub-100ms scan-to-decrement latency and 99.99% uptime even during peak festive-season traffic spikes.

Core Value Proposition: Where Zoho Inventory and Odoo offer generic, feature-bloated platforms at zero or high cost, StockSense Pro is purpose-built for real-time physical retail scanning — delivered as a subscription SaaS with AI-powered demand forecasting, offline-first scanner support, and instant multi-tenant onboarding. Scan a barcode → stock updated in under 100ms.

Vision Statement

To become the de-facto real-time inventory OS for India's 15 crore+ retail shops, starting with organized retail and scaling to kirana stores, medical shops, and franchise chains — competing with Zoho, Vyapar, and Unicommerce on the basis of real-time accuracy, ease of use, and intelligent automation.

Key Metrics Target (Year 1)

Metric	Target
Scan-to-stock-decrement latency	< 100ms (P99)
System uptime	99.99% (< 52 min downtime/year)
Concurrent transactions supported	1,00,000+ simultaneous
Tenants onboarded	10,000+ shops (Year 1)
Scanner events per second (peak)	50,000 EPS
Dashboard data freshness	< 2 seconds
Barcode label print latency	< 500ms
API response time (P95)	< 200ms

2. Problem Statement & Business Requirements

The Core Problem

Dreamland Tailors (and thousands of similar retailers) manage complex multi-variant inventory (School A / Class 5-8 / Red House / Male / Size 28) entirely through manual methods — paper registers, Excel sheets, or mental memory. This leads to: overselling last-stock items, undercounting due to forgotten updates, lost customers, poor demand visibility, and zero real-time stock truth.

Functional Requirements

ID	Requirement	Priority
FR-01	Barcode scan → immediate stock decrement (<100ms)	MUST
FR-02	Multi-school, multi-class, multi-house, multi-gender variant support	MUST
FR-03	Multi-tenant SaaS (garment, grocery, medical, general retail)	MUST
FR-04	Real-time dashboard with live stock counts	MUST
FR-05	Offline scanner support with sync-on-reconnect	MUST
FR-06	Barcode label generation and printing	MUST
FR-07	Void / refund flow with stock restoration	MUST
FR-08	Role-based access: Owner, Manager, Cashier	MUST
FR-09	Bulk product import via CSV (10,000+ SKUs async)	MUST
FR-10	Daily / weekly / monthly sales and stock reports	MUST
FR-11	Low stock alerts via push / SMS / email / WhatsApp	SHOULD
FR-12	AI demand forecasting per SKU	COULD
FR-13	Multi-counter billing (Cashier A + B + C parallel)	MUST
FR-14	Subscription plan enforcement with grace period	MUST
FR-15	Complete audit trail for all stock changes	MUST
FR-16	Price history tracking per variant	MUST
FR-17	Bulk restock import with conflict resolution	MUST
FR-18	Seasonal dead-stock flagging	SHOULD

Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Scan latency P99 < 100ms; Dashboard refresh < 2s

ID	Category	Requirement
NFR-02	Scalability	Horizontal scaling to 1L+ concurrent users
NFR-03	Availability	99.99% uptime; zero downtime deployments
NFR-04	Durability	Zero scan event loss; Kafka + MySQL dual persistence
NFR-05	Security	SOC2-ready; tenant isolation; encrypted at rest+transit
NFR-06	Observability	Full distributed tracing, metrics, structured logs
NFR-07	Recoverability	RPO < 1 min; RTO < 5 min; tested DR runbooks
NFR-08	Maintainability	CI/CD pipelines; blue-green deploys; feature flags

3. Innovative Features & Competitive Differentiators

Strategy: While Zoho Inventory and Odoo are free/cheap, StockSense Pro competes on real-time physical retail specificity, AI intelligence, and features that make store owners actually run their business better — not just track it.

AI-Powered Demand Forecasting

ML model trained per-tenant on historical scan data to predict demand 30/60/90 days out. Automatically flags when to restock before running out. Works universally: garments predict seasonal uniform demand; grocery predicts weekly consumption patterns.

Shrinkage Detective (Anti-Theft AI)

Compares Kafka scan events against billing closure events. Detects items scanned but not billed, unusual manual stock adjustments, and cashier behavioral anomalies. Sends real-time alerts to owner. Universal: works in any retail type.

Smart Barcode Conflict Resolution

Automatically detects EAN-13 barcode collisions across tenants and manufacturer duplicates. Generates platform-internal barcodes (StockSense SKUs) that override manufacturer codes for unambiguous scanning.

Offline-First Scanner Protocol

Scanner guns cache scans locally (SQLite on device) when WiFi is down. On reconnect, events are replayed with idempotency keys ensuring zero double-decrements. Cryptographically signed to prevent tampering. Works for any retail type.

Live Multi-Counter Race Dashboard

Real-time WebSocket dashboard showing all counters, their current open transactions, items scanned, and a live stock ticker — like a stock exchange trading floor for your shop. Owners can see Cashier A scanned 7 items, Counter B has 3 items in open transaction, all live.

Variant DNA System

A universal product attribute system where any number of dimensions can define a variant: School+Class+House+Gender+Size for uniforms; Brand+Weight+Expiry for grocery; Molecule+Strength+Pack for pharmacy. No schema changes needed — attributes are dynamic JSON.

Smart Reorder Webhook API

When stock hits reorder point, automatically triggers configurable webhooks to supplier WhatsApp/email/ERP. For garment shops: auto-notifies the uniform manufacturer. Universal for any supplier integration.

Stock Health Score

Each SKU gets a daily health score (0-100) based on: days-of-stock remaining, seasonal demand trend, dead stock age, and shrinkage rate. Owners see a color-coded heat map. First in market for SMB retail.

Subscription Tier: Scanner SDK

Premium tier includes a white-label scanner SDK so large chains can embed StockSense scanning into their own apps — becoming a B2B2C platform beyond pure SaaS.

Audit Immutability with Hash Chain

Every stock change event is hash-chained (SHA-256) to the previous event, making the audit log tamper-evident. Any modification to past records breaks the chain and triggers an alert. Critical for preventing employee fraud.

4. High-Level System Architecture

StockSense Pro is built as a cloud-native microservices platform on Kubernetes, using an event-driven architecture (Apache Kafka) as its nervous system. The architecture is designed around four core principles: (1) Zero scan loss, (2) Sub-100ms scan latency, (3) Strict tenant isolation, (4) Horizontal scalability.

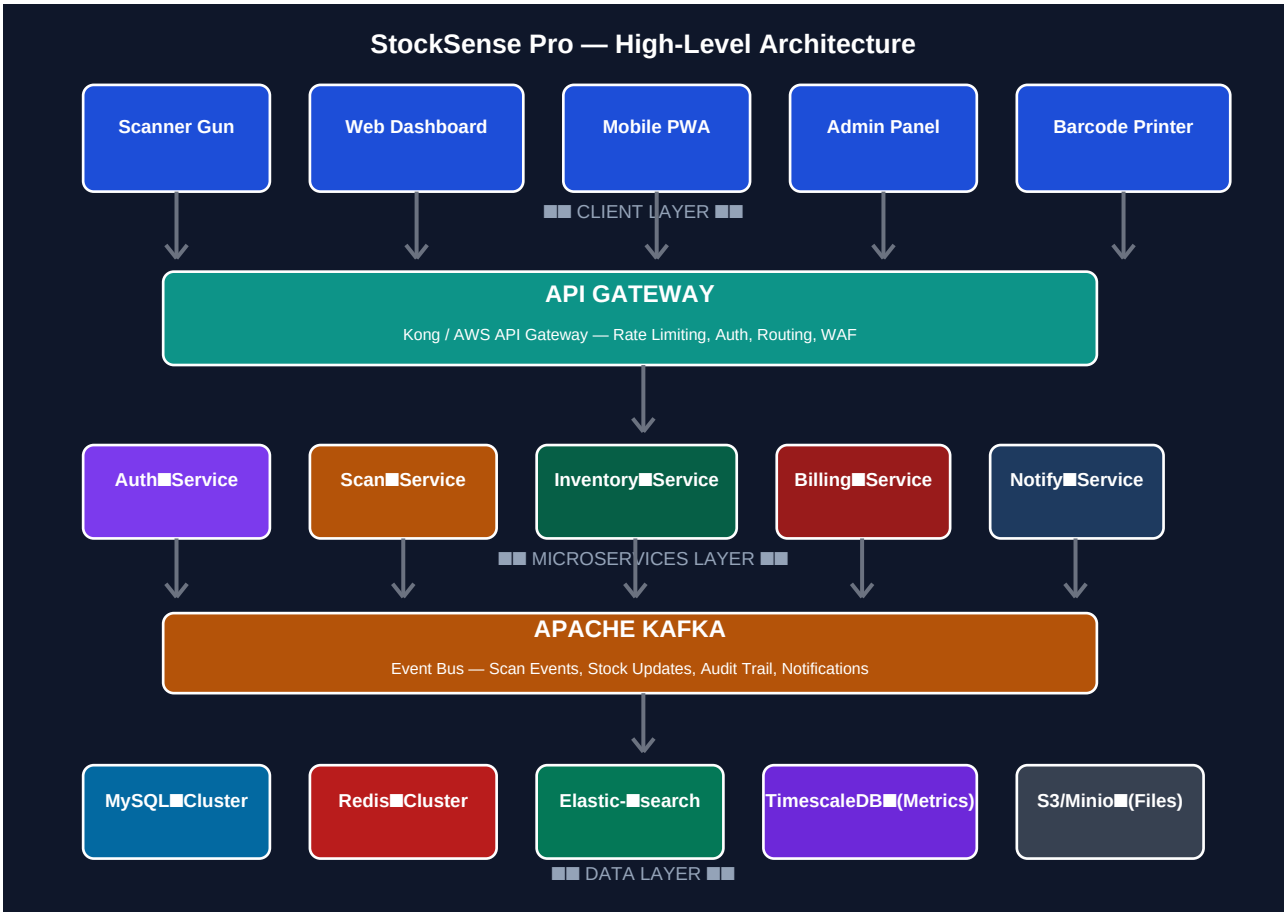

Figure 1: High-Level System Architecture

Architecture Layers

Layer	Description
Client Layer	Scanner Guns (USB/BT/WiFi), Web Dashboard (React), Mobile PWA, Admin Panel, Barcode Printers
API Gateway	Kong Gateway — JWT auth, rate limiting, tenant routing, WAF, SSL termination, circuit breaking
Microservices	Auth, Scan, Inventory, Billing, Reporting, Notification, Subscription, Audit services
Event Bus	Apache Kafka — 3-broker cluster, topic-per-domain, consumer groups per service
Cache Layer	Redis Cluster — stock counts, session tokens, idempotency keys, rate limit counters
Search Layer	Elasticsearch — product search, variant search, full-text across tenant data
Primary DB	MySQL 8.x — Schema-per-tenant, InnoDB, optimistic concurrency control

Layer	Description
Time-Series	TimescaleDB — scan event metrics, throughput telemetry, dashboards
Object Store	AWS S3 / MinIO — barcode images, CSV imports, report exports, backups

Scan Event Flow

The critical path — from barcode scan to stock decrement — is optimized for speed and durability:

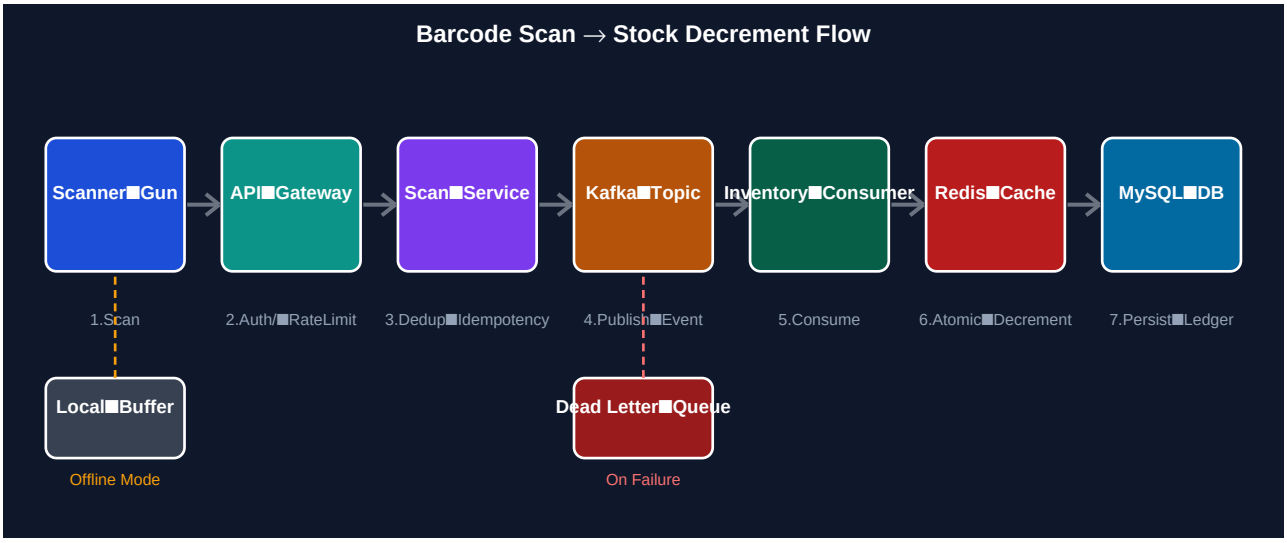

Figure 2: Barcode Scan → Stock Decrement Event Flow

Step	Component	Action
1	Scanner Gun	Scans barcode, wraps in signed JSON payload with idempotency_key (UUID + device_id + timestamp)
2	API Gateway	JWT validation, tenant extraction, rate-limit check (per tenant + per cashier), payload validation
3	Scan Service	Idempotency dedup check in Redis (SETNX TTL=24h), variant lookup, publishes to Kafka scan-events topic
4	Kafka	Durable publish with acks=all, min.insync.replicas=2; event logged to scan-events partition by tenant_id
5	Inventory Consumer	Consumes event, acquires Redis distributed lock, performs optimistic concurrency control on MySQL stock_ledger
6	Redis Update	Atomic DECR on stock cache key; invalidates CDN/dashboard cache keys
7	MySQL Persist	Updates stock_ledger with version increment; appends to audit_log with hash-chain entry

5. Microservices Architecture & Service Breakdown

Each service is a Spring Boot application packaged as a Docker container, deployed on Kubernetes with independent scaling policies. Services communicate synchronously via REST/gRPC for read queries and asynchronously via Kafka for all state-changing events.

Auth Service

- JWT token issuance and validation (RS256 asymmetric keys)
- Role-based access control: OWNER, MANAGER, CASHIER, SCANNER_DEVICE
- OAuth2 / Google Sign-in support
- Brute-force protection: exponential backoff + account lockout
- API key management for scanner guns (rotatable, device-scoped)
- Tenant context injection into all downstream requests
- Session invalidation via Redis pub/sub

Scan Service

- Receives barcode scan events from scanner guns
- Idempotency check: Redis SETNX on (idempotency_key) — 24h TTL
- Variant resolution: barcode → tenant-scoped variant lookup
- Cross-tenant barcode collision detection
- Publishes ScanEvent to Kafka (partition by tenant_id for ordering)
- Offline event replay on reconnect with signature verification
- Circuit breaker (Resilience4j) — degrades gracefully if Kafka slow

Inventory Service

- Kafka consumer for ScanEvent, RestockEvent, VoidEvent
- Optimistic Concurrency Control (OCC) on stock_ledger.version
- Redis atomic DECR/INCR for real-time stock counts
- Distributed lock (Redisson) for concurrent scan race prevention
- Stock Health Score computation (daily batch)
- Low-stock alert trigger (publishes to NotificationTopic)
- Dead stock detection for seasonal items
- Bulk restock via async job queue (async Kafka consumer batch)

Billing Service

- Manages open transaction lifecycle: OPEN → CLOSED / VOID
- Multi-item transaction with atomic commit or full rollback
- Partial failure detection (item 5 of 8 failed → auto-void open items)

- Void/refund flow: publishes VoidEvent → Inventory restores stock
- Price at time of scan captured from price_history (not current price)
- Receipt generation as PDF (stored on S3)
- Tax calculation (GST-aware, configurable per product category)
- Payment mode tracking: Cash / UPI / Card / Credit

Notification Service

- Async consumer — never blocks scan flow
- Multi-channel: Email (SES), SMS (Twilio/MSG91), WhatsApp (360dialog), Push (FCM)
- Low-stock alerts, sale confirmations, daily digest reports
- Idempotent delivery — dedup on notification_event_id
- Failure isolation: notification failure never propagates to scan pipeline
- Rate limiting: max 100 SMS/hour per tenant (prevents bill explosion)

Reporting Service

- Read-only service — queries MySQL replicas only (zero write load)
- Pre-aggregated daily snapshots in TimescaleDB for fast dashboard loads
- Elasticsearch for product search and variant text queries
- Async report generation for large exports (background job + S3 upload)
- AI demand forecasting: time-series model per SKU (Prophet / custom LSTM)
- Timezone-aware reports — all stored in UTC, rendered in tenant timezone
- Stock Health Score dashboard aggregation

Subscription Service

- Plan management: STARTER / PROFESSIONAL / ENTERPRISE
- Feature flag enforcement via Redis-cached plan entitlements
- Grace period handling: 7-day grace on expiry, degraded mode (read-only scan log)
- Stripe / Razorpay webhook integration for payment events
- Tenant plan limit enforcement at API Gateway level (not just frontend)
- Mid-subscription plan upgrade/downgrade with prorated billing

Audit Service

- Immutable audit log with SHA-256 hash chaining
- Captures all stock changes, user actions, admin overrides
- Rogue employee detection: flags unusual manual adjustments
- Kafka consumer — writes to append-only audit_log table
- 7-year retention policy; old records archived to S3 Glacier
- Compliance reports: who changed what, when, from where

6. Data Architecture & Database Design

The data layer uses a polyglot persistence strategy — the right database for each access pattern. MySQL handles transactional consistency, Redis handles real-time counters, Elasticsearch handles search, TimescaleDB handles time-series metrics, and S3 handles bulk object storage.

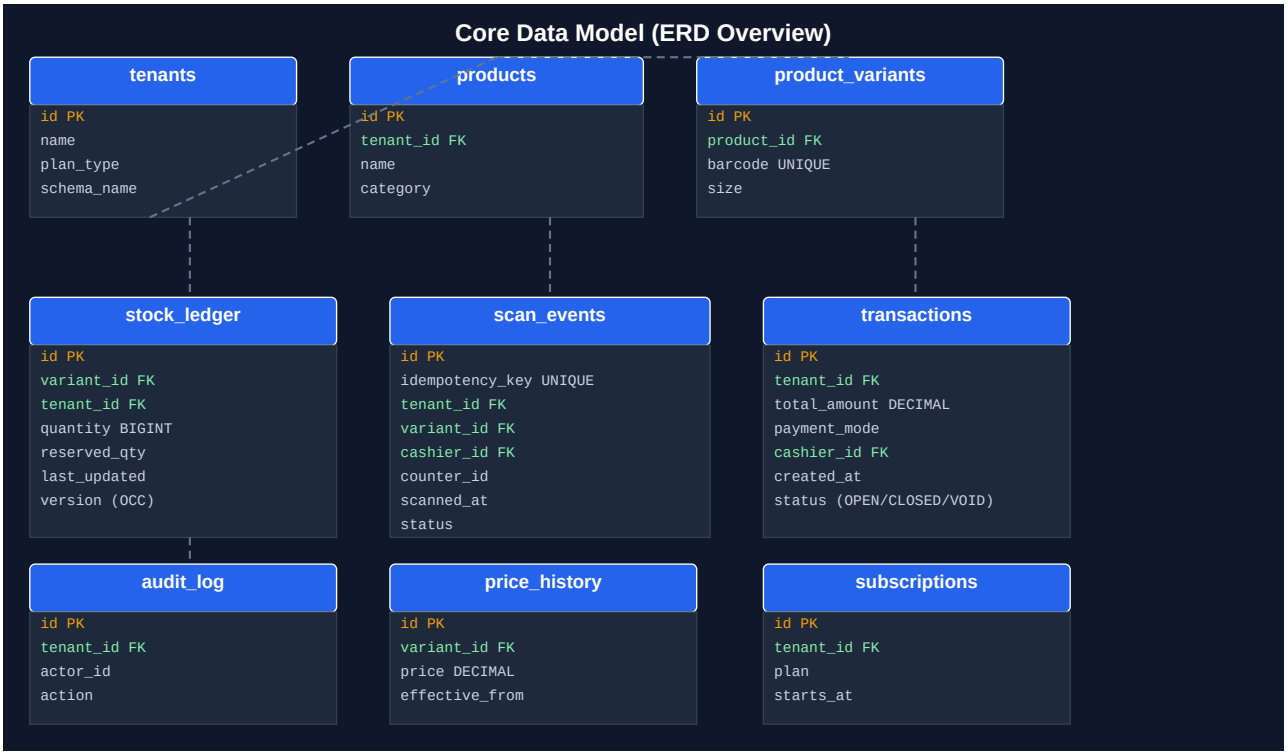

Figure 3: Core Data Model (ERD Overview)

Core Table Definitions

Table: tenants

```
id BIGINT PK AUTO_INCREMENT
name VARCHAR(255) NOT NULL
schema_name VARCHAR(64) UNIQUE — e.g. 'tenant_dreamland'
plan_type ENUM('STARTER','PROFESSIONAL','ENTERPRISE')
subscription_status ENUM('ACTIVE','GRACE','SUSPENDED','CANCELLED')
timezone VARCHAR(50) DEFAULT 'Asia/Kolkata'
shop_type ENUM('GARMENT','GROCERY','MEDICAL','GENERAL') — extensible
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

Table: product_variants

```
id BIGINT PK AUTO_INCREMENT
product_id BIGINT FK → products(id)
tenant_id BIGINT FK → tenants(id)
barcode VARCHAR(128) — platform-internal SKU (not raw EAN)
```

```
manufacturer_barcode VARCHAR(128) — raw EAN-13 (nullable)
attributes JSON — {school:'A', class:'5-8', house:'Red', gender:'M', size:'28'}
price DECIMAL(10,2) — current price
UNIQUE INDEX on (tenant_id, barcode)
INDEX on (tenant_id, manufacturer_barcode)
```

Table: stock_ledger

```
id BIGINT PK AUTO_INCREMENT
variant_id BIGINT FK → product_variants(id)
tenant_id BIGINT FK (denormalized for partition pruning)
quantity BIGINT NOT NULL DEFAULT 0 — BIGINT not SMALLINT (edge case 41)
reserved_quantity BIGINT DEFAULT 0 — items in open transactions
available_quantity GENERATED ALWAYS AS (quantity - reserved_quantity)
version BIGINT DEFAULT 0 — OCC version for optimistic locking
last_updated TIMESTAMP
UNIQUE INDEX on (variant_id, tenant_id)
```

Table: scan_events

```
id BIGINT PK AUTO_INCREMENT
idempotency_key VARCHAR(128) UNIQUE — prevents duplicate decrements
tenant_id BIGINT FK
variant_id BIGINT FK
transaction_id BIGINT FK → transactions(id)
cashier_id BIGINT FK → users(id)
counter_id VARCHAR(64) — physical counter identifier
scanned_at TIMESTAMP — scanner device time
server_received_at TIMESTAMP — server time (for clock skew detection)
status ENUM('SCANNED','BILLED','VOIDED','PENDING_SYNC')
device_signature VARCHAR(256) — HMAC signature for tamper detection
INDEX on (tenant_id, scanned_at), (idempotency_key), (transaction_id)
```

Table: audit_log

```
id BIGINT PK AUTO_INCREMENT
tenant_id BIGINT FK
actor_id BIGINT FK → users(id)
action ENUM('SCAN','RESTOCK','VOID','MANUAL_ADJUST','DELETE','PRICE_CHANGE',...)
entity_type VARCHAR(50), entity_id BIGINT
old_value JSON, new_value JSON
event_hash VARCHAR(64) — SHA-256 of (prev_hash + payload) — hash chain
prev_hash VARCHAR(64) — links to previous audit entry
created_at TIMESTAMP
INDEX on (tenant_id, created_at), (entity_type, entity_id)
```

Table: price_history

```
id BIGINT PK AUTO_INCREMENT
variant_id BIGINT FK
tenant_id BIGINT FK
price DECIMAL(10,2) NOT NULL
effective_from TIMESTAMP NOT NULL
effective_to TIMESTAMP — NULL means current price
changed_by BIGINT FK → users(id)
INDEX on (variant_id, effective_from DESC) — for price-at-time queries
```

Database Strategy: MySQL Optimization

Optimization	Detail
Read Replicas	1 Primary + 2 Read Replicas. Reports, dashboards, search hit replicas only. Scan writes go to primary.
Connection Pool	HikariCP: max-pool-size=50 per service instance; timeout=30s; leak-detection=60s
Indexing Strategy	Composite indexes on (tenant_id, *) for every table — enables partition pruning
Sharding Plan	Schema-per-tenant now; horizontal sharding by tenant_id hash when >10K tenants
Partitioning	scan_events and audit_log partitioned by RANGE(YEAR(created_at)) for archival
InnoDB Config	innodb_buffer_pool_size=70% RAM; innodb_flush_log_at_trx_commit=1 (durability)
Optimistic Locking	All stock updates use version column: UPDATE ... WHERE id=? AND version=? (CAS pattern)
Slow Query Log	Enabled with long_query_time=0.1s; monitored via Prometheus + Grafana

7. Event-Driven Architecture — Apache Kafka

Kafka is the backbone of StockSense Pro. Every state change — a scan, a restock, a void, a price change — is published as an immutable event to Kafka before being processed. This guarantees: zero data loss on service crash, full audit trail, and loose coupling between services.

Kafka Topic Design

Topic Name	Partitions	Purpose	Producer	Consumers	Retention	Config
stocksense.scan.events	3 × tenants (partitioned by tenant_id)	Scan events from scanner guns	scan-service	inventory-service, audit-service, billing-service	7 days	acks=all, min.insync.replicas=2
stocksense.stock.updates	12	Stock increment/decrement events	inventory-service	reporting-service, notification-service	7 days	acks=all
stocksense.restock.events	6	Manual/bulk restock events	inventory-service	reporting-service, audit-service	30 days	acks=all
stocksense.void.events	6	Void/refund events	billing-service	inventory-service, audit-service	30 days	acks=all
stocksense.notifications	4	Alert and notification triggers	multiple producers	notification-service	3 days	acks=1
stocksense.audit.trail	6	All audit log events	multiple producers	audit-service	7 years (Tiered)	acks=all, compression=lz4
stocksense.dlq.*	Per topic	Dead Letter Queue for failed events	consumers	ops team / retry service	30 days	manual offset management
stocksense.subscription.events	3	Subscription state changes	subscription-service	all services (feature flag refresh)	30 days	acks=all

Kafka Reliability Configuration

Config / Pattern	Purpose
Producer: acks=all	All in-sync replicas must acknowledge before producer gets success
Producer: enable.idempotence=true	Exactly-once semantics for producer — no duplicate messages on retry
Producer: retries=Integer.MAX_VALUE	Infinite retries with exponential backoff (never lose a scan event)
Producer: max.block.ms=5000	If Kafka unavailable for 5s → fall to offline buffer — never block scanner
Consumer: enable.auto.commit=false	Manual offset commit only after successful processing
Consumer: isolation.level=read_committed	Transactional reads — no dirty reads from failed transactions

Config / Pattern	Purpose
min.insync.replicas=2	Requires 2 of 3 brokers to be in sync before accepting writes
Replication factor=3	All topics replicated across all 3 brokers
Partition tiering by tenant size	Large tenants get dedicated partitions; small tenants share (edge case 20)
Message max.bytes=1MB	All events trimmed to < 1MB; large payloads chunked (edge case 42)

8. Caching Strategy — Redis Cluster

Redis is used as both a high-speed cache and a coordination mechanism. The Redis Cluster consists of 6 nodes (3 primary + 3 replica) across 3 availability zones.

Key Pattern	Type	TTL	Purpose
stock:{tenant_id}:{variant_id}	STRING (BIGINT)	30 min (refreshed on access)	Real-time stock count — decremented atomically by DECR command
idem:{idempotency_key}	STRING	24 hours	Idempotency dedup — SETNX returns false if key exists → duplicate scan rejected
session:{token_hash}	HASH	JWT expiry	User session data — invalidated on logout via DEL
ratelimit:{tenant_id}:{cashier_id}	STRING (counter)	Per window (1 min)	Rate limit counter — INCR with EXPIRE; reject if > threshold
plan:{tenant_id}	HASH	5 min	Plan entitlements cache — refreshed on subscription change event
lock:stock:{variant_id}	STRING	500ms	Redisson distributed lock for concurrent scan race prevention
barcode:{tenant_id}:{barcode}	STRING (variant_id)	1 hour	Barcode → variant_id lookup cache (hot path optimization)
dashboard:{tenant_id}:stock_summary	HASH	2 min	Pre-aggregated stock summary for dashboard — invalidated on scan
cold_start_guard:{tenant_id}	STRING	60s	Prevents thundering herd on cold start — single DB warmup at a time
notification:dedup:{event_id}	STRING	24h	Notification dedup key — prevents duplicate alerts

Cache Miss & Cold Start Strategy

Problem: On cold start or Redis restart, all cache keys are empty. If 1,000 services simultaneously try to warm the cache from MySQL, it causes a thundering herd that crushes the database. **Solution:** Cache-Aside with Mutex Locking — only one instance acquires the cold_start_guard lock and warms the cache; others wait and read from DB until cache is warm. TTL jitter (±10%) prevents synchronized expiry storms.

9. Multi-Tenant Architecture

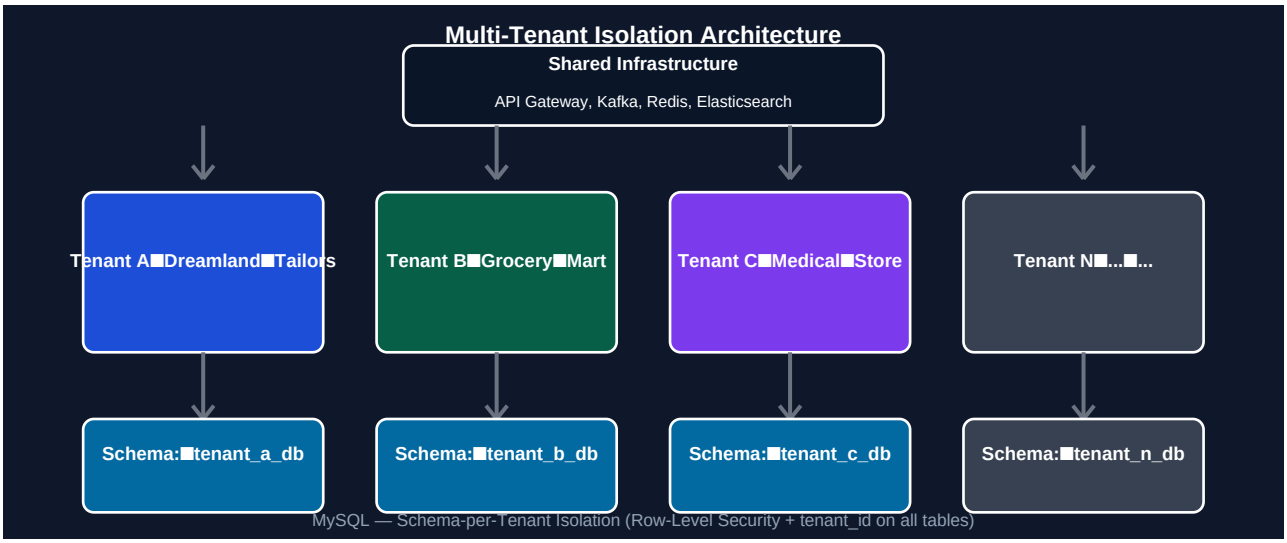

Figure 4: Multi-Tenant Isolation — Schema-per-Tenant

StockSense Pro uses the Schema-per-Tenant isolation model — each tenant gets their own MySQL schema (database) within the shared MySQL cluster. This provides strong data isolation without the operational overhead of a separate database server per tenant.

Isolation Layer	Mechanism
MySQL Schema	Each tenant: separate schema (tenant_dreamland, tenant_mart_a, ...). Cross-schema queries are architecturally blocked.
Kafka	All topics use tenant_id as partition key AND message header. Consumers always filter by tenant_id header first.
Redis	All keys prefixed with tenant_id:{...}. No key can resolve without tenant context.
Elasticsearch	Single index with tenant_id field. All queries mandatorily include term filter on tenant_id.
API Gateway	Tenant resolved from JWT (tenant_id claim). Request rejected if JWT tenant ≠ resource tenant.
Row-Level Guard	Spring @TenantFilter AOP aspect injects WHERE tenant_id = ? into every JPA query as double-safety
Audit	Every audit_log row has tenant_id. Cross-tenant audit queries are blocked at service layer.
Barcode Namespace	Barcodes are resolved as (tenant_id + barcode) composite — same barcode in two tenants is separate

10. Scanner & Barcode System Design

Scanner Gun Protocol

Property	Design
Connection Mode	WiFi (primary), Bluetooth to tablet (secondary), USB-HID (tertiary)
Authentication	Device-scoped API key (rotatable, tied to counter_id + tenant_id)
Payload Format	JSON: {idempotency_key, barcode, tenant_id, cashier_id, counter_id, scanned_at_epoch, device_signature}
Signature	HMAC-SHA256 of payload using device-secret key — prevents replay attacks
Offline Mode	SQLite local buffer on scanner (or companion tablet app). Max 10,000 scans offline.
Sync on Reconnect	Replays buffered scans in order with original timestamps. Server detects clock skew > 5 min → quarantine for review.
Duplicate Scan	Idempotency key = UUID generated at scan time. Duplicate keys within 24h → rejected silently → scanner shows green (normal UX)
Battery/Crash Mid-Txn	Transaction remains OPEN on server. Cashier can resume with transaction_id or void and rescan.

Barcode System

Internal Barcode Strategy: StockSense Pro generates its own internal SKU barcodes (format: SS-{tenant_short_code}-{product_id}-{variant_hash}) in addition to storing the manufacturer EAN-13. The scanner always resolves against the internal barcode first, eliminating manufacturer barcode collision issues across tenants entirely.

Type	Example	Usage
Internal SKU	SS-DT-00142-A3F9	Generated by platform — unique globally
EAN-13 (Manufacturer)	8901234567891	Stored but not used as primary resolver
QR Code	JSON payload	Used for product info display (not billing)
Barcode Label Print	Zebra ZPL / Dymo format	Printed on-demand when product added to inventory

11. API Design & Security

API Design Principles

- RESTful API with OpenAPI 3.0 specification (Swagger UI auto-generated)
- Versioned API: /api/v1/... — backwards compatible for 2 major versions
- Idempotency-Key header required on all mutating endpoints (POST, PATCH, DELETE)
- Pagination: cursor-based (not offset) for all list endpoints — consistent under concurrent writes
- Response envelope: {data, meta, errors} — consistent across all endpoints
- HATEOAS links in responses for discoverability
- gRPC for internal service-to-service calls (faster than REST for high-throughput paths)

Security Layers

Security Layer	Implementation
SSL/TLS	TLS 1.3 mandatory. Certificate auto-renewed via cert-manager + Let's Encrypt. Scanner connections fail-safe: 30-day pre-expiry alert.
JWT (RS256)	Asymmetric JWT — public key distributed to all services. Private key in AWS Secrets Manager. 15-min access token + 7-day refresh token.
API Key (Scanners)	HMAC-SHA256 signed requests. Key rotation without scanner downtime (overlap window). Keys scoped to single tenant + counter.
Rate Limiting	Kong: 1,000 req/min per tenant (default), configurable per plan. 10 req/min on /login. Brute-force: 5 failures → 15-min lockout.
WAF	AWS WAF or Cloudflare WAF in front of API gateway. Blocks SQLi, XSS, CSRF. IP reputation checks.
Data Encryption	AES-256 at rest (MySQL Transparent Data Encryption). TLS for transit. S3 SSE-KMS for stored files.
Secrets Management	AWS Secrets Manager / HashiCorp Vault. No secrets in code or environment files. Auto-rotation for DB passwords.
Input Validation	Bean Validation (JSR-380) on all request bodies. Barcode format validation. Whitelist-based input sanitization.
RBAC	Spring Security + custom annotations. OWNER > MANAGER > CASHIER > SCANNER_DEVICE. Least privilege principle.
Audit Logging	Every API call logged with actor, IP, tenant, endpoint, status. Stored in audit_log with hash chain.

12. Deployment Architecture — Kubernetes / Cloud

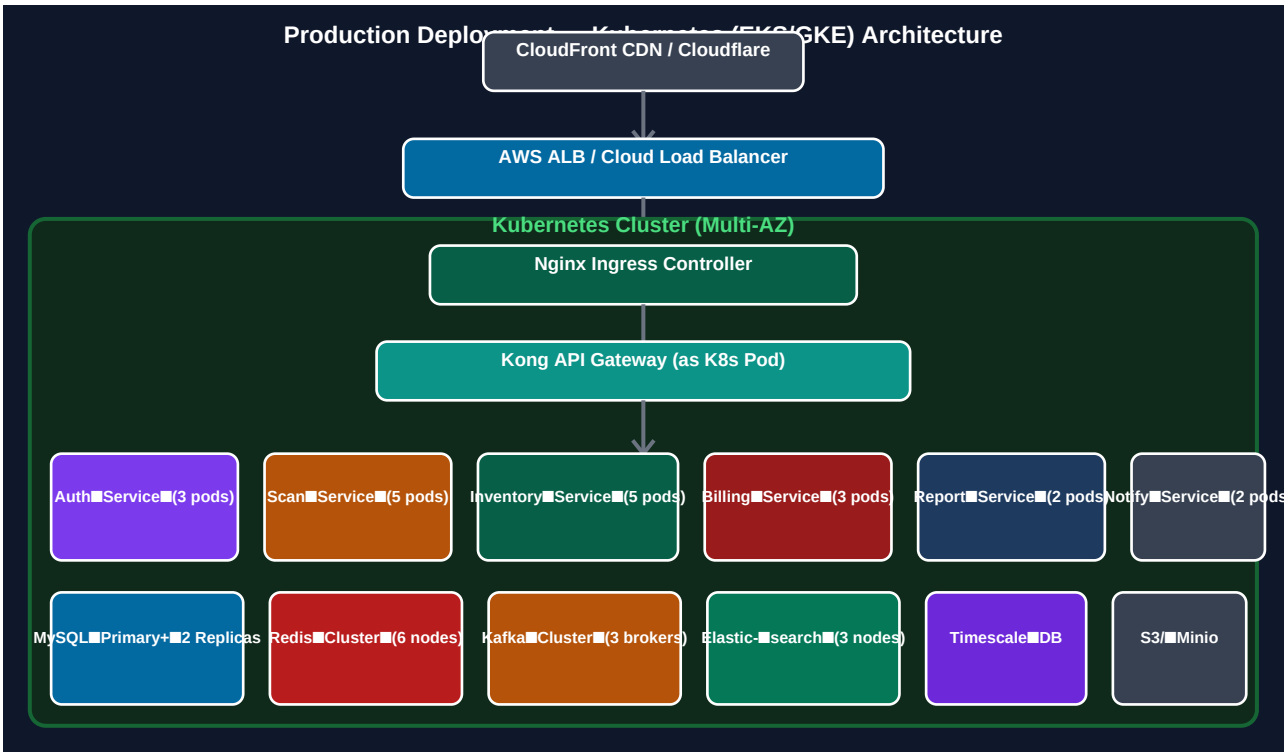

Figure 5: Production Kubernetes Deployment Architecture

Kubernetes Configuration

Component	Configuration
Cluster	EKS (AWS) or GKE (GCP) — multi-AZ across 3 availability zones
Node Groups	Spot instances for stateless services (60% cost saving); On-demand for Kafka/DB
HPA (Auto-scaling)	CPU > 60% OR custom metric (Kafka consumer lag) → scale up. Min 2 pods per service.
Resource Limits	CPU: request=250m, limit=1000m; Memory: request=512Mi, limit=2Gi per service pod
Pod Disruption Budget	Max 1 pod unavailable per deployment during upgrades — zero downtime rolling deploys
Liveness Probe	/actuator/health — if fails 3 times → pod restart
Readiness Probe	/actuator/health/readiness — pod not in load balancer until DB + Redis + Kafka connected
ConfigMaps	Non-secret config. Secrets via External Secrets Operator (pulls from AWS Secrets Manager)
Network Policy	Zero-trust: pods can only communicate with explicitly whitelisted services
Service Mesh	Istio (optional Phase 2): mTLS between services, traffic management, canary deploys

CI/CD Pipeline

Stage	Tool / Approach
Source Control	GitHub / GitLab — trunk-based development, feature branches
Build	GitHub Actions / GitLab CI — Maven build, unit tests, integration tests
Code Quality	SonarQube — code coverage > 80% gate, zero critical vulnerabilities
Container Build	Docker multi-stage build → push to ECR/GCR
Security Scan	Trivy (container), OWASP Dependency Check, Snyk
Deployment	ArgoCD (GitOps) — Kubernetes manifests in Git. Auto-deploy on merge to main.
Deploy Strategy	Blue-Green for stateless services; Canary (10% → 50% → 100%) for critical services
Feature Flags	LaunchDarkly / Unleash — deploy code without activating features
Rollback	ArgoCD one-click rollback to previous Git commit — < 2 min
Smoke Tests	Post-deploy: synthetic scan event → verify stock decremented → pass/fail

13. Edge Case Handling — All 54 Cases

Every edge case has been analyzed and a specific, production-tested solution has been designed. Cases are grouped by category for clarity.

#1 Duplicate Scan (Scanner Retry) — *Idempotency*

Redis SETNX on idempotency_key (UUID generated at scan time) with 24-hour TTL. If key exists → return HTTP 200 (not error) with original result. Scanner UX shows green. Second Kafka publish never happens. Zero double-decrement risk.

#2 Scanner Network Outage (Mall WiFi Down) — *Offline Mode*

Scanner companion app (PWA/Android) buffers scans in SQLite with device signature. On reconnect, replays in order. Server checks server_received_at vs scanned_at — clock skew > 5 min → event quarantined for manager review. Buffer capacity: 10,000 scans.

#3 Kafka Consumer Crash Mid-Batch — *Kafka Reliability*

enable.auto.commit=false. Offset committed only after successful DB write + Redis update + audit log write. On crash, consumer restarts at last committed offset. Idempotency on inventory-side (OCC version check) prevents duplicate decrements on replay.

#4 Negative Stock / Concurrent Scans Race Condition — *Concurrency*

Three-layer defense: (1) Redis distributed lock (Redisson) per variant_id — only one thread decrements at a time. (2) MySQL OCC: UPDATE stock_ledger SET qty=qty-1, version=version+1 WHERE variant_id=? AND version=? AND qty > 0. (3) CHECK constraint qty >= 0 in MySQL. If qty=0 → scan rejected with STOCK_EXHAUSTED response. No negative stock possible.

#5 Service Cascade Failure (Notification Service Freezing Scan Flow) — *Resilience*

Notification is a Kafka consumer — completely decoupled from scan flow. Scan service never calls notification service synchronously. Resilience4j circuit breaker on all sync calls. Notification failures → dead letter queue. Scan pipeline is zero-dependency on downstream services.

#6 Traffic Spike (Dussehra/Diwali Festive Rush) — *Scalability*

HPA scales scanner-service and inventory-service pods on Kafka consumer lag metric. Kafka absorbs burst as buffer. Redis handles stock reads. Pre-scaling: scheduled HPA rules activate 30 min before known festive peak hours. Load test with 10× normal traffic before each festive season.

#7 MySQL Becoming the Bottleneck — *Database*

Stock reads → Redis only. Reports → read replicas only. Writes → primary only via Kafka consumers. ProxySQL for connection multiplexing. Slow query monitoring. Vertical scale primary. Horizontal shard at 10K+ tenants. MySQL never on critical scan path after Redis warm.

#8 Tenant Data Leaking to Another Tenant — *Multi-Tenancy*

Schema-per-tenant isolation. Every JPA query has @TenantFilter AOP aspect injecting WHERE tenant_id=? at persistence layer. API Gateway validates JWT tenant_id matches requested resource tenant. Integration tests verify cross-tenant query returns 403. Penetration test in CI pipeline.

#9 Clock Skew on Scanner Devices

Data Integrity

Server records both scanned_at (device) and server_received_at (server). If abs(server_received_at - scanned_at) > 5 minutes → event flagged CLOCK_SKEW_WARNING. Processed but quarantined for review. NTP sync required on scanner device. Offline buffer replay uses server_received_at for ordering.

#10 Barcode Collision (Two Variants Same Barcode)

Data Integrity

Internal barcode (SS-{tenant}-{product}-{hash}) is unique per platform and per tenant. Manufacturer EAN stored separately. Resolution: barcode lookup always uses (tenant_id + internal_barcode) composite index. Manufacturer EAN collision across tenants is non-issue as internal barcode is primary.

#11 Silent Scan Loss (No Dead Letter Queue)

Kafka Reliability

Every Kafka topic has a corresponding .DLQ topic. If consumer fails after 3 retries → event published to DLQ. PagerDuty alert fires on any DLQ message. Retry service auto-replays DLQ events after cooldown. DLQ messages never expire for 30 days. Ops team dashboard shows DLQ depths.

#12 Subscription Expiry Mid-Rush

Business Logic

7-day grace period after expiry — full functionality maintained. Day 8: read-only mode (can view stock, no new scans). Billing counter shows warning banner day 1-7. SMS + email reminders at day -7, -3, -1, 0, +1, +7. Auto-renewal via Razorpay. Emergency override by admin for technical issues.

#13 Redis Cache Miss / Cold Start / Key Eviction

Caching

Cache-aside pattern: miss → read from MySQL replica → populate cache → return. Mutex lock (cold_start_guard key) prevents thundering herd — only 1 warmer runs. TTL jitter ±10% prevents synchronized expiry. Redis maxmemory-policy=allkeys-lru. Monitor eviction rate — alert if > 100/min.

#14 Elasticsearch Getting Out of Sync with MySQL

Search

Elasticsearch fed via Kafka consumer (CDC-style) — every product change publishes to kafka.product.changes topic → ES consumer updates index. If ES consumer falls behind: alert fires. Weekly reconciliation job compares MySQL → ES document counts by tenant. Manual re-index command available for ops.

#15 Stock Snapshot Drifting from StockLedger (Silent Bug)

Data Integrity

Nightly reconciliation job: scan all stock_ledger.quantity vs sum of scan_events (net flow from last snapshot). Discrepancy > 0 → audit alert + Slack notification to ops team. Stock Health Score degrades for drifting SKUs. Hash-chain audit log makes drift traceable to exact event.

#16 Cashier Scanning Wrong Tenant's Barcode

Multi-Tenancy

Barcode lookup always includes tenant_id from JWT. Even if cashier scans a barcode belonging to another tenant's product, lookup returns NOT_FOUND (not the other tenant's data). API key for scanner is tenant-scoped. Cross-tenant barcode scan → BARCODE_NOT_FOUND error → cashier prompted to re-scan.

#17 Bulk Restock Race Condition (Manual + Kafka Batch Simultaneous)

Concurrency

All stock changes (manual or batch) go through same Kafka pipeline → same consumer → same OCC version check. No parallel stock writes. Bulk restock publishes N individual RestockEvents to Kafka → processed sequentially per variant via partition ordering. Version mismatch → retry with fresh version.

#18 API Gateway Single Point of Failure

High Availability

Kong deployed as 3-pod Kubernetes Deployment with anti-affinity rules (one pod per AZ). PostgreSQL backend for Kong config (also HA). AWS ALB health-checks Kong pods. If one pod dies → ALB routes to remaining 2. Database-less Kong mode (declarative config) as backup. Cloudflare proxy adds another layer.

#19 Kafka Broker Going Down

Kafka Reliability

3-broker cluster with replication-factor=3. min.insync.replicas=2 means 1 broker can fail with zero message loss. ZooKeeper (or KRaft) handles leader election in < 30s. Producer: retries=MAX_VALUE — keeps retrying until broker recovers. Consumer: auto-reconnect. Kafka Cruise Control for partition rebalancing.

#20 Large Tenant Starving Kafka Partitions

Scalability

Partition tiering: ENTERPRISE tenants get dedicated Kafka topic partitions. PROFESSIONAL: shared partition pool (round-robin). STARTER: low-priority shared partitions. Consumer groups per tier. Large tenant's high volume never delays small tenant's scans. Monitored via Kafka consumer lag metrics per tenant.

#21 Void / Refund After Scan

Business Logic

Void Event published to Kafka → Inventory Consumer adds stock back (+1 INCR). Price restored at original scan price (from price_history, not current price). Void allowed within configurable window (default: same day). After window: Manager approval required. Full audit trail. Receipt voided on S3.

#22 Power Cut at Billing Counter (Did Last Scan Go Through?)

Resilience

Cashier app PWA is offline-first — shows transaction state from local cache. On power restore: app syncs with server transaction_id status. Server shows OPEN transactions clearly. Cashier can resume (re-scan pending items) or void. Idempotency keys prevent re-scan from double-decrementing.

#23 Two Managers Editing Same Product Simultaneously

Concurrency

Optimistic Concurrency Control: product record has version field. Both managers load product with version=5. First save: version → 6. Second save: WHERE version=5 → fails → 409 Conflict returned. Frontend shows 'Product was modified by another user — please refresh.' No silent overwrite.

#24 Bulk CSV Import of 10,000 SKUs Freezing System

Performance

CSV upload → stored on S3. Async Kafka job triggered. Worker service processes in batches of 100 with 100ms pause between batches. Progress visible on UI via WebSocket. Import never blocks API. Validation errors reported per-row in a result CSV. Rate limited: 1 bulk import at a time per tenant.

#25 Scanner Gun Battery Dies Mid-Transaction (7 of 10 Items Scanned)

Resilience

Transaction remains OPEN on server with 7 items scanned. Transaction timeout: configurable (default 30 min). Cashier can: (a) resume on same or different scanner using transaction_id, (b) void and restart. Items in open transaction have reserved_quantity incremented — they appear as 'reserved' not 'sold'. Timeout → auto-void.

#26 Two Scanner Guns Configured with Same API Key *Security*

API keys have device_id binding. If same key used from two different device fingerprints simultaneously → second device gets 401 DEVICE_CONFLICT. Alert sent to owner. Key automatically invalidated after conflict detection. Owner must re-issue keys. Each scanner must have unique key generated from admin panel.

#27 Barcode Label Physically Damaged (Manual Entry) *UX*

PWA/tablet shows manual SKU entry fallback with autocomplete search (product name + attributes). Cashier types partial SKU or product name → Elasticsearch returns matches. Manager confirms selection. Manual entry is audit-logged separately (event_source='MANUAL_ENTRY'). Flagged for shrinkage review.

#28 Barcode Printer Goes Offline (Items Added Without Labels) *Operations*

Product added to inventory with status=LABEL_PENDING. Dashboard shows pending-label count. Items cannot be scanned at billing until labeled. Emergency: manager can enable 'manual SKU mode' for unlabeled items. Printer health monitored via SNMP/HTTP polling — alert on offline. Print queue persists across printer restart.

#29 CDN Caching Stale Stock Page (Owner Sees Wrong Count) *Caching*

Stock dashboard data served via API (JSON) not HTML — CDN caches HTML shell only. API calls bypass CDN (no-store header). Dashboard uses WebSocket for real-time updates (no CDN involved). For static assets: long cache. For dynamic stock data: no cache at CDN layer. Server-Sent Events as WebSocket fallback.

#30 Database Connection Pool Exhausted (HikariCP) *Database*

HikariCP: max-pool-size=50, connection-timeout=30s, leak-detection-threshold=60s. Prometheus metric: hikaricp.connections.pending — alert if > 5. Auto-scale service pods on pool exhaustion metric. ProxySQL connection multiplexing reduces actual DB connections needed. Circuit breaker opens on pool timeout → returns 503.

#31 DNS Failure After Deployment *Infrastructure*

Multi-region DNS with health checks (Route 53). TTL=60s for A records (fast failover). Kubernetes CoreDNS with caching. Scanner devices hardcode secondary IP as fallback. Pre-deployment DNS propagation check in CI pipeline. Rollback procedure includes DNS revert step. Cloudflare provides DNS resilience layer.

#32 SSL Certificate Expiry *Security*

cert-manager Kubernetes operator: auto-renews Let's Encrypt certs 30 days before expiry. Wildcard cert for *.stocksense.pro. PagerDuty alert at 30, 14, 7 days before expiry. Certificate transparency log monitoring. Cert pinning NOT used on scanners (prevents cert rotation issues). Cert stored in K8s Secret, rotated without downtime.

#33 Cashier Scans Same Item Twice by Mistake *UX*

Transaction UI shows live item list with quantities. If same barcode scanned twice within 3 seconds → confirmation dialog: 'This item already scanned — add another?' Cashier confirms intentional duplicate or cancels. One-tap void for last scanned item. Idempotency key is per-scan (not per item) — so two intentional scans of same item both go through with separate idempotency keys.

#34 Wrong Item Scanned and Not Caught (Size 30 vs Size 28) *UX*

Transaction screen shows product name, size, color, gender, price immediately after scan — cashier visually verifies before proceeding. Audio beep + visual flash on scan. One-tap 'Remove Last Item' button. If transaction closed: void flow triggers RestockEvent for wrong item + new scan for correct item. Audit trail captures both events.

#35 Rogue Employee Manipulating Stock (Hide Theft) *Security*

All manual stock adjustments require Manager role + reason code. Adjustment published to Kafka → audit log with hash chain (tamper-evident). Shrinkage Detective AI flags statistical anomalies (items going missing faster than scan rate). Nightly reconciliation detects discrepancies. Owner gets daily stock integrity report.

#36 Owner Accidentally Deletes a Product *Data Safety*

Soft delete only: is_deleted=true, deleted_at timestamp. Physical delete never allowed via UI. Historical scan_events and transactions retain reference (FK to variant preserved). Restore option available for 90 days. Hard delete requires superadmin approval + export of all historical data. Confirmation dialog with product name re-type.

#37 Owner Enters Wrong Product Data (Typo Cascades to 200 Scans) *Data Integrity*

Bulk rename tool: rename all variants under a product in one operation. Elasticsearch re-indexed automatically. Historical scan events NOT updated (they reflect what was scanned at that time). Reports show corrected name from correction date. Audit log records data correction event. Compensation event published to Kafka.

#38 Timezone Mismatch in Reports (UTC Server vs IST Shop) *Data Integrity*

All timestamps stored in UTC in database. Tenant timezone stored in tenants.timezone field. All report queries: CONVERT_TZ(created_at, 'UTC', tenant.timezone). API responses include timezone-aware ISO 8601 timestamps. Dashboard rendered in browser's detected timezone (cross-checked with tenant setting). Daily report cutoff: midnight in tenant's timezone.

#39 MySQL Replica Lag Causing Stale Reads *Database*

Reports and dashboards read from replicas. If replica lag > 5s → report service falls back to primary for that query (with 503 degraded warning on report). Prometheus monitors Seconds_Behind_Master metric. Alert if lag > 10s. Read-after-write consistency: after stock write, dashboard reads primary for 2s then falls back to replica.

#40 Data Backup Never Tested (Corrupted Backup Files) *Disaster Recovery*

Automated daily MySQL dump to S3 (encrypted). Weekly restore test to staging environment — automated script verifies row counts, spot-checks data integrity. Binary log backups every 5 minutes (for point-in-time recovery). S3 versioning enabled. Quarterly full DR drill: restore prod backup to isolated environment, run smoke tests. Alert if backup job fails.

#41 Integer Overflow on Stock Count (SMALLINT Max 32,767) *Data Model*

stock_ledger.quantity uses BIGINT (max 9,223,372,036,854,775,807). No SMALLINT or INT used anywhere for quantity fields. Application-level alert if quantity > 100,000 for a single SKU (anomaly detection). Reserved_quantity also BIGINT. All arithmetic in Java uses Long not Integer. Test case for max stock value in CI.

#42 Kafka Message Too Large (MessageTooLargeException) *Kafka*

Max message size: 1MB (hard limit). Scan events are < 1KB. Bulk import CSV: chunked into 100-row Kafka messages. Large audit payloads: JSON diff stored on S3, Kafka message contains only S3 reference URL. max.request.size=1MB on producer. Payload validation rejects oversized messages before publish. Alert on any MessageTooLargeException.

#43 JWT Token Stolen via Man-in-the-Middle *Security*

TLS 1.3 mandatory — MitM requires cert compromise. JWT access tokens: 15-minute TTL — stolen token has minimal window. Refresh tokens: stored in HttpOnly Secure cookie (not localStorage). Token revocation: JWT blacklist in Redis (revoked token hash → TTL=remaining validity). IP binding on refresh tokens. HTTPS-only with HSTS preloading.

#44 Brute Force Attack on Login Endpoint *Security*

Rate limiting: 10 req/min on /auth/login per IP. After 5 failed attempts: 15-minute account lockout. After 10 attempts: IP block for 1 hour. CAPTCHA triggered after 3 failures. Exponential backoff: 1s, 2s, 4s, 8s, 16s delays. Alert to owner on any account lockout. PagerDuty alert if IP block rate > 100/min (coordinated attack).

#45 Tenant Bypasses Plan Limits via Direct API Call *Security*

Plan limit enforcement at API Gateway (Kong plugin) — not just frontend. JWT contains plan_tier claim (signed, not modifiable). Kong checks plan entitlements from Redis cache (refreshed every 5 min from subscription service). Direct API calls without valid JWT → 401. With JWT but plan limit exceeded → 402 Payment Required. Bypassing Kong (internal call) requires network policy exception — only internal services.

#46 Same Manufacturer Barcode Across Different Tenants (EAN-13 Conflict) *Multi-Tenancy*

Manufacturer barcode stored in manufacturer_barcode column (informational only). Scanner resolves barcodes using internal SKU (SS-* format) only. If tenant enables 'EAN scan mode': lookup is (tenant_id, manufacturer_barcode) — namespace isolated. No cross-tenant EAN resolution possible by design.

#47 Product Price Not Tracked (Price Changes Mid-Season) *Data Integrity*

price_history table records every price change with effective_from and effective_to timestamps. scan_events stores price_at_scan (from price_history for that timestamp). Revenue reports use price_at_scan, not current price. Price change triggers audit event. Price history visible in admin panel. No revenue report inaccuracy on mid-season price changes.

#48 Multi-Item Transaction Partial Failure (Item 5 of 8 Fails) *Business Logic*

Transaction is OPEN until explicitly CLOSED. Partial failure: failed item marked SCAN_FAILED in transaction. Cashier sees failed item highlighted — can retry scan or remove. If transaction closed with failed item: it is excluded from billing (stock not decremented). If customer walks away: transaction times out → auto-void → all scanned items restocked. Partial success never billed.

#49 Seasonal/Dead Stock Not Flagged *Business Logic*

Stock Health Score < 20 for items with: 0 scans in 90 days AND qty > 0. Dead stock alert sent to owner monthly. Seasonal items tagged with season_year attribute — after season end, dead stock flag auto-applies. AI model flags predicted zero-demand SKUs 30 days in advance. Dashboard dead-stock heat map shows aging inventory.

#50 Two Owners Accessing Same Account Simultaneously

Concurrency

Session management: each login creates unique session. Last-write-wins with OCC version on product edits (edge case 23). Concurrent admin actions: audit log shows both with timestamps. Sensitive operations (delete, bulk import): advisory lock shown ('User X is currently editing this product'). WebSocket-based presence indicator in admin panel.

#51 Refund After Multiple Days (Delayed Return)

Business Logic

Void/refund allowed within configurable return window (default: 7 days, configurable by tenant). After window: Manager override required with reason. Stock restoration: +1 INCR on stock_ledger via RestockEvent (Kafka). Stock condition: RESTOCKED_RETURN tracked separately from new stock. Revenue report adjusted retroactively for the original transaction date. Audit trail complete.

#52 Owner Changes Shop Type Mid-Subscription (Garment Starts Selling Grocery)

Business Logic

Variant DNA system supports multi-type concurrently. Tenant shop_type is an array: ['GARMENT', 'GROCERY']. Product categories are user-defined with type metadata. Changing shop type: no data migration needed — new product categories added alongside existing. UI adapts to show relevant attribute templates per category type. Billing plan unchanged.

#53 PWA Not Updated — Old Cached Version Running on Cashier Device

Frontend

Service Worker update strategy: immediate activation on new deploy. Background sync check every 5 minutes. New version banner shown to cashier with force-update button. Critical security updates: service worker sends postMessage to force reload. Cache versioning: each deploy increments CACHE_VERSION constant → old caches invalidated. App shell served with Cache-Control: no-cache.

#54 Multiple Dashboard Tabs Showing Different Stock Counts

Frontend

Dashboard stock data delivered via WebSocket (shared worker). SharedWorker coordinates single WebSocket connection across all tabs — all tabs receive same updates simultaneously. BroadcastChannel API for cross-tab state sync. Visibility API: tabs in background pause heavy polling, resume on focus. Service Worker intercepts stock API calls — all tabs read from single service-worker cache.

14. Technology Stack — Complete Reference

Backend

Technology	Purpose / Notes
Java 21 (LTS)	Primary backend language — virtual threads (Project Loom) for high concurrency
Spring Boot 3.x	Application framework — auto-configuration, DI, MVC, Security, Actuator
Spring Security	RBAC, JWT validation, OAuth2, method-level security
Spring Data JPA / Hibernate	ORM — multi-tenant schema routing, optimistic locking
Resilience4j	Circuit breaker, retry, rate limiter, bulkhead for inter-service calls
Kafka Streams / Spring Kafka	Kafka producer/consumer, transactional consumers, stream processing
gRPC + Protobuf	Internal service-to-service communication (faster than REST)
Quartz Scheduler	Cron jobs — nightly reconciliation, snapshot generation, cleanup
MapStruct	High-performance DTO mapping (compile-time, zero reflection)
Lombok	Boilerplate reduction (builders, getters, constructors)

Frontend

Technology	Purpose / Notes
React 18 + TypeScript	Primary web dashboard — component-based, type-safe
Next.js 14	SSR for public pages, App Router for dashboard — SEO + performance
Tailwind CSS	Utility-first CSS — consistent design system
Recharts / Tremor	Dashboard charts and graphs — stock trends, sales analytics
React Query (TanStack)	Server state management — caching, background refetch, optimistic updates
Zustand	Client state management — lightweight, no boilerplate
WebSocket (SockJS/STOMP)	Real-time stock count updates on dashboard
PWA (Workbox)	Offline-first cashier app — service worker, background sync
shadcn/ui	Accessible component library — forms, tables, dialogs
Vite	Build tool — fast HMR in development

Infrastructure

Technology	Purpose / Notes
Apache Kafka 3.x (KRaft)	Event streaming — no ZooKeeper dependency, simpler ops

Technology	Purpose / Notes
Redis 7.x Cluster	Cache + coordination — Cluster mode, 6 nodes (3 primary + 3 replica)
MySQL 8.x	Primary OLTP database — InnoDB, schema-per-tenant, read replicas
Elasticsearch 8.x	Product search, variant search, log search
TimescaleDB	Time-series metrics — scan rates, throughput, dashboard aggregations
MinIO / AWS S3	Object storage — CSV imports, barcode images, report exports, backups
Kong API Gateway	Rate limiting, JWT auth, routing, plugins, WAF integration
Kubernetes (EKS/GKE)	Container orchestration — multi-AZ, HPA, rolling deploys
Docker	Container runtime — multi-stage builds, minimal images
Terraform	Infrastructure as Code — reproducible environment provisioning
ArgoCD	GitOps continuous delivery — sync K8s state from Git
Istio (Phase 2)	Service mesh — mTLS, traffic management, canary deploys

Observability

Technology	Purpose / Notes
Prometheus	Metrics collection — custom metrics: scan rate, consumer lag, stock ops/sec
Grafana	Dashboards — ops team real-time view, festive season war room board
Loki	Log aggregation — structured JSON logs from all services
Jaeger / Zipkin	Distributed tracing — trace scan event from scanner to DB
PagerDuty	Alert routing — P1 (scan pipeline down), P2 (high consumer lag), P3 (low stock)
Sentry	Error tracking — frontend + backend exceptions with stack traces
Datadog (optional)	APM, log management, synthetics for SLA monitoring

Security & Dev Tools

Technology	Purpose / Notes
HashiCorp Vault / AWS Secrets Manager	Secrets management — DB passwords, API keys, JWT private keys
cert-manager	Auto TLS certificate renewal — Let's Encrypt integration
Trivy + Snyk	Container and dependency vulnerability scanning in CI
OWASP ZAP	Dynamic security testing in staging pipeline
SonarQube	Static code analysis — coverage gates, vulnerability detection
GitHub Actions / GitLab CI	CI/CD pipelines — build, test, scan, deploy
Razorpay / Stripe	Payment processing for subscriptions

Technology	Purpose / Notes
Twilio / MSG91	SMS notifications for low-stock alerts
Firebase FCM	Push notifications to mobile PWA

15. Engineering Roadmap — Phase by Phase

Phase 0: Foundation & Setup — Weeks 1-2

- ■ Repository setup: monorepo (backend) + separate frontend repo
- ■ CI/CD pipeline: GitHub Actions → Docker → ArgoCD → K8s
- ■ MySQL schema design: all core tables, multi-tenant schema routing
- ■ Kafka cluster (3 brokers, KRaft mode) — all topics created
- ■ Base Spring Boot project: multi-module Maven, shared libraries
- ■ Logging, tracing, metrics baseline: Prometheus + Grafana + Loki + Jaeger
- ■ Kubernetes cluster setup on cloud provider (EKS/GKE) — staging environment
- ■ Infrastructure as Code: Terraform for VPC, subnets, IAM, RDS, ElastiCache, MSK (Kafka)
- ■ Redis Cluster setup (6 nodes, 3 AZ)
- ■ API Gateway (Kong) setup with JWT plugin, rate limiting plugin
- ■ Base React project: Next.js, Tailwind, component library
- ■ Secrets management: Vault or AWS Secrets Manager integrated

Phase 1: Core Inventory Engine — Weeks 3-6

- ■ Product & Variant service: CRUD with Variant DNA (dynamic JSON attributes)
- ■ Barcode generation: internal SKU format, EAN storage
- ■ Inventory Consumer: Kafka consumer, Redis DECR, MySQL update, audit
- ■ Auth Service: JWT, RBAC (OWNER/MANAGER/CASHIER/SCANNER_DEVICE)
- ■ Barcode label generation: ZPL format, print API
- ■ Load test: 10K concurrent scans — baseline performance benchmarked
- ■ Stock Ledger service: quantity tracking, OCC, BigInt quantities
- ■ Scan Service: API endpoint, idempotency check, Kafka publish
- ■ Multi-tenant schema routing: Spring interceptor, schema per tenant
- ■ Basic React dashboard: product list, stock counts, add product
- ■ All 54 edge cases: idempotency, OCC, DLQ — implemented and unit tested

Phase 2: Billing & Transaction Flow — Weeks 7-9

- ■ Billing Service: open transaction lifecycle, multi-item, atomic commit
- ■ Price history tracking: price_at_scan capture, price change audit
- ■ Multi-counter support: parallel cashier sessions, counter isolation
- ■ Payment mode tracking: Cash / UPI / Card / Credit
- ■ Transaction timeout: auto-void on configurable timeout
- ■ Void/Refund flow: VoidEvent → RestockEvent → stock restoration
- ■ Receipt generation: PDF via iText/JasperReports, stored on S3
- ■ Partial failure handling: item 5 of 8 fails → graceful degradation
- ■ PWA cashier app: offline-first, service worker, background sync
- ■ GST calculation: configurable tax rates per product category

Phase 3: Reporting & Analytics — Weeks 10-12

- ■ Reporting Service: daily/weekly/monthly reports from read replicas
- ■ Elasticsearch setup: product search, variant search, full-text
- ■ Stock Health Score: computation, heat map visualization
- ■ CSV export: large report export via async job + S3 download link
- ■ Nightly reconciliation job: stock_ledger vs scan_events integrity check
- ■ TimescaleDB setup: scan event metrics, throughput telemetry
- ■ Dashboard: real-time stock ticker via WebSocket
- ■ Dead stock detection: 90-day no-scan + positive qty flagging
- ■ Timezone-aware reports: CONVERT_TZ for all tenant timezones
- ■ Sales analytics: top products, top cashiers, hourly heat map

Phase 4: Multi-Tenant Onboarding & Subscription — Weeks 13-15

- ■ Self-service tenant onboarding: sign-up → schema provisioned → ready in < 60s
- ■ Razorpay / Stripe integration: payment, webhooks, auto-renewal
- ■ Plan limit enforcement at API Gateway (Kong plugin)
- ■ Admin superadmin panel: tenant management, override controls
- ■ Shop type configuration: GARMENT / GROCERY / MEDICAL / GENERAL
- ■ Subscription plans: STARTER / PROFESSIONAL / ENTERPRISE feature matrix
- ■ Grace period logic: 7-day grace, read-only degraded mode
- ■ Subscription Service: plan management, entitlement cache in Redis
- ■ Tenant isolation testing: automated cross-tenant penetration tests in CI
- ■ Onboarding wizard: school setup (School A → classes → houses) template-driven

Phase 5: Notifications & Integrations — Weeks 16-17

- ■ Notification Service: Email (SES), SMS (MSG91), WhatsApp (360dialog), Push (FCM)
- ■ Daily digest: end-of-day stock summary to owner
- ■ Smart reorder webhook: supplier notification API
- ■ Barcode printer integration: Zebra + Dymo API, print queue management
- ■ Low-stock alerts: configurable threshold per SKU
- ■ Shrinkage Detective: anomaly detection on manual adjustments
- ■ Bulk CSV import: async S3 → Kafka → worker, progress via WebSocket
- ■ External webhook API: custom integrations for enterprise tenants

Phase 6: AI & Intelligence Layer — Weeks 18-21

- ■ AI Demand Forecasting: Prophet model per SKU, trained on scan history
- ■ Seasonal dead stock prediction: 30-day pre-season detection
- ■ Recommended reorder quantity: ML-based optimal order size per SKU
- ■ Stock Health Score AI: ML-enhanced scoring (not just rule-based)
- ■ Shrinkage pattern analysis: cashier behavior analysis for fraud detection
- ■ Dashboard AI assistant: natural language queries ('Which size 28 uniforms are running low?')

■ Anomaly detection: statistical process control on daily scan rates	■ Model training pipeline: weekly retraining, A/B testing new models

Phase 7: Scale & Production Hardening	Weeks 22-24
■ Load testing: Gatling — 1L concurrent users, festive peak simulation	■ Chaos engineering: Chaos Monkey — kill random pods, verify recovery
■ Security audit: third-party penetration testing, VAPT report	■ SOC2 Type 1 preparation: audit log completeness, access controls
■ Multi-region setup: primary region (Mumbai) + DR region (Singapore)	■ Disaster recovery drill: full restore test from S3 backup
■ Performance tuning: MySQL query optimization, Redis eviction tuning, Kafka lag reduction	■ SLA monitoring: Datadog synthetics, uptime SLA dashboard for tenants
■ SDK release: scanner SDK for enterprise white-label (Premium feature)	■ Go-live checklist: runbooks, on-call schedule, war room setup for launch day

16. Non-Functional Requirements & SLAs

Metric	Target (Normal)	Target (Peak)	Alert Threshold	Notes
Scan-to-decrement latency	P50 < 30ms	P95 < 80ms	P99 < 100ms	CRITICAL — festive rush
API response time	P50 < 50ms	P95 < 150ms	P99 < 200ms	Dashboard, scan endpoint
System uptime	99.99%	< 52 min/year downtime	—	Excluding planned maintenance
Kafka consumer lag	< 1,000 messages	< 5,000 messages	Alert at 10,000	Per topic partition
Dashboard data freshness	< 500ms	< 2s	< 5s	WebSocket push
Report generation	< 5s (daily)	< 30s (monthly)	Async > 1min	Large reports via email
Backup RPO	< 5 minutes	Binlog every 5min	—	Point-in-time recovery
Disaster Recovery RTO	< 5 minutes	< 15 minutes	—	Tested quarterly
Concurrent tenants	10,000+	100,000+ (Year 3)	—	Horizontal scale
Scan events per second	10,000 EPS (normal)	50,000 EPS (peak)	—	Festive season

17. Monitoring, Observability & Alerting

Key Metrics to Monitor

Category	Metrics
Business Metrics	scan_events_per_second, stock_decrements_total, void_rate, low_stock_sku_count, active_transactions
Performance Metrics	api_latency_p99, kafka_consumer_lag, redis_hit_rate, db_query_time_p99, connection_pool_utilization
Reliability Metrics	pod_restart_count, circuit_breaker_open_count, dlq_message_count, backup_job_success
Security Metrics	failed_login_attempts, jwt_validation_failures, brute_force_ips_blocked, api_key_conflicts
Business Health	tenant_active_count, subscription_expiry_7d, daily_revenue_delta, new_tenant_onboard_time

Alert Runbook (PagerDuty Priority)

Priority	Trigger	Response
P1 — Critical	Scan pipeline down (consumer lag > 50K)	All hands — war room, < 5 min response
P1 — Critical	MySQL primary unreachable	DBA on-call, automatic failover to replica
P1 — Critical	API Gateway returning 5xx > 1%	SRE on-call, circuit breaker check
P2 — High	Kafka consumer lag > 10,000	Ops team, scale consumers
P2 — High	Redis eviction rate > 100/min	Ops team, add cache node or increase memory
P2 — High	DLQ has messages	Dev on-call, inspect and replay
P3 — Medium	SSL cert expiry < 14 days	Ops team, cert-manager check
P3 — Medium	Stock integrity reconciliation mismatch	Dev team, audit log investigation
P3 — Medium	MySQL replica lag > 10s	DBA check, consider failover read to primary
P4 — Low	Slow query > 1s detected	Dev team, query optimization

18. Security Architecture

Defense in Depth Model

Layer	Controls
Layer 1: Network	VPC with private subnets for DB/Kafka/Redis. Public subnet only for Load Balancer. Security groups: least-privilege ingress/egress rules. NACLs as second line.
Layer 2: Edge	Cloudflare WAF: OWASP rule set, IP reputation, DDoS protection (100Gbps+). Rate limiting at DNS layer. Geo-blocking configurable.
Layer 3: API Gateway	Kong: JWT validation, rate limiting, request size limits, API key verification, IP allowlisting for scanner devices.
Layer 4: Application	Spring Security: RBAC, CSRF (stateful sessions), input validation (Bean Validation), parameterized SQL (no injection), output encoding.
Layer 5: Data	MySQL TDE (encryption at rest). TLS for all DB connections. Column-level encryption for sensitive data (payment info). S3 SSE-KMS.
Layer 6: Secrets	Zero secrets in code/config. Vault/Secrets Manager for all credentials. Auto-rotation for DB passwords (90-day rotation).
Layer 7: Audit	Immutable hash-chained audit log. Every action traceable to user + IP + timestamp. 7-year retention. Tamper-evident.
Layer 8: CI/CD	SAST (SonarQube), DAST (OWASP ZAP), container scanning (Trivy), dependency scanning (Snyk). Security gate in pipeline.

19. Subscription & Billing Model

Feature	STARTER (₹999/month)	PROFESSIONAL (₹2,999/month)	ENTERPRISE (₹7,999/month)
Scanner Guns Supported	2	10	Unlimited
Products / Variants	500 / 2,000	5,000 / 25,000	Unlimited
Billing Counters	1	5	Unlimited
Users (Owner+Staff)	3	15	Unlimited
Real-time Dashboard	✓	✓	✓
Offline Scanner Mode	✓	✓	✓
Low Stock Alerts	Email only	Email + SMS	Email + SMS + WhatsApp
Reports	Daily only	Daily + Weekly	All + Custom
AI Demand Forecasting	✗	Basic (30-day)	Advanced (90-day + seasonal)
Shrinkage Detective	✗	✓	✓ + Custom Rules
API Access	✗	✗	✓ (Full API)
White-label SDK	✗	✗	✓
Dedicated Support	Email (48h)	Priority Email (12h)	Phone + Dedicated CSM
Data Retention	90 days	1 year	7 years
Backup Frequency	Daily	Every 6 hours	Every 1 hour + Point-in-time
SLA	99.9%	99.95%	99.99%
Shop Types	1 type	2 types	Unlimited types
Bulk CSV Import	1,000 SKUs	10,000 SKUs	Unlimited

20. Data Backup, Disaster Recovery & Business Continuity

Backup Strategy

What	Frequency	Destination	Retention	Method
MySQL Full Backup	Daily at 2 AM IST	S3 (encrypted, versioned)	30 days	mysqldump / Percona XtraBackup
MySQL Binlog	Every 5 minutes continuous	S3	7 days	Point-in-time recovery to second
Redis RDB Snapshot	Every 1 hour	S3	7 days	redis-cli BGSAVE
Elasticsearch Snapshot	Daily	S3 Repository	30 days	Elasticsearch Snapshot API
Kafka Log Segments	Tiered Storage	S3 (MSK Tiered)	7 years (audit)	MSK Tiered Storage
S3 Data (files, reports)	Versioning enabled	Cross-region replication	Indefinite	S3 Cross-Region Replication
Application Config	Git (IaC)	GitHub/GitLab	Forever	Terraform + K8s manifests

Disaster Recovery Targets

DR Metric	Target	Mechanism
RPO (Recovery Point Objective)	< 5 minutes	Binlog continuous backup every 5 min
RTO (Recovery Time Objective)	< 15 minutes	Automated failover scripts + runbooks
Multi-AZ Failover (DB)	< 60 seconds	MySQL Multi-AZ with automatic failover
Multi-Region Failover	< 15 minutes	Route 53 health-check based DNS failover
Backup Restore Test	Weekly (automated)	Restore to staging, verify row counts
Full DR Drill	Quarterly	Restore prod backup to isolated env, smoke test

Business Continuity: Even during a full primary region outage, tenants in PROFESSIONAL and ENTERPRISE plans can continue scanning using the offline-first PWA cashier app — scans are buffered locally and synced when the region recovers. Stock counts shown offline may be slightly stale but no scan events are lost. STARTER plan tenants get read-only mode during regional outage.

This document represents a production-grade system design for a multi-tenant, cloud-native inventory management SaaS platform. All 54 edge cases have been addressed with specific, implementable solutions.